



BITS Pilani

Introduction to AI Systems

Dr. Minati Rath
mrath@wilp.bits-Pilani.ac.in



<SS ZG662, Introduction to AI Systems>

Session 1 **These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.**

- Objective of course
- Evaluation Plan
- Course Overview

This course is an introductory course to AI Systems.

Discuss the architecture, components, and ecosystem of modern AI/ML systems.

Design and analyze data and feature pipelines for AI applications

Evaluate model development workflows, including training, fine-tuning, validation, and model selection.

Assess the deployment, operation, and governance of AI systems and distinguish AI/ML systems from traditional software systems from both engineering and operational perspectives



At the end of this session, participants should be able to:

Differentiate AI models from AI systems.

Understand the lifecycle of production AI systems.

Identify critical components of enterprise AI platforms.

Analyze why many AI projects fail despite having accurate models.

Evaluate AI opportunities within their organizations.



Every successful AI initiative begins with a **business objective**, not a machine learning model.

The first question is:

"What business problem are we trying to solve?"

Examples:

Reduce customer churn

Detect fraudulent transactions

Improve product recommendations

Automate document processing

Forecast demand accurately

Once the problem is defined, evaluate whether it can be solved using **explicit business rules**.

Business Problem



Can clear rules solve it?



—► Rule-Based Solution



Need to learn patterns from data?



Machine Learning Solution

Option 1: Rule-Based Solution

Choose a rule-based approach when:

Rules are clear and deterministic.

Logic rarely changes.

Domain experts can define decision criteria.

Explanations are important.

Examples

IF order value > \$10,000

THEN require manager approval

IF age < 18

THEN reject application

Characteristics

Fast implementation

Easy to understand

Easy to audit

No training data required

Option 2: Machine Learning Solution

Choose ML when:

Rules are difficult to define.

Patterns are hidden in data.

Decisions depend on many variables.

The environment changes over time.

Examples

Customer Churn Prediction

Instead of: IF complaints > 3 AND

purchases < 2 THEN customer will leave

Use historical customer data to learn complex relationships automatically.

Fraud Detection

Fraudsters constantly change behavior, making static rules ineffective. ML learns evolving patterns from transaction history.

"Not every business problem needs AI. The most mature organizations first ask whether a simple rule-based solution can achieve the desired outcome. Machine Learning becomes valuable when the complexity of the problem exceeds what humans can reliably encode as rules."

Why do AI projects fail?



Is it due to

- Poor models
- Lack of data
- Lack of talent

A large percentage of failures occur because organizations focus on models while neglecting

system integration,
governance,
deployment,
monitoring,
retraining,
and business alignment.

Key Concept



A model is only one component.

Real-world AI systems include:

Business Problem



Data Pipeline



Feature Engineering



Model Development



Deployment



Monitoring



Business Process Integration

Example - Credit Risk Prediction

Building a model may take 20% effort.

The remaining 80% involves:

Data integration

Compliance

Explainability

Monitoring

Workflow integration

AI, Machine Learning and Deep Learning in Enterprise Context



AI - Broad capability enabling machines to perform intelligent tasks.

Examples:

- Route optimization
- Fraud detection
- Intelligent document processing
- Enterprise search

Machine Learning –

Systems learn patterns from data.

Examples:

Function	Use Case
Banking	Credit scoring
Telecom	Churn prediction
Manufacturing	Predictive maintenance
Retail	Demand forecasting

Deep Learning - Used when data complexity increases.

Examples:

Medical image analysis
Speech recognition
Autonomous vehicles
Large Language Models

Not every business problem requires Deep Learning.

Organizations often obtain greater ROI from: Logistic Regression
Gradient Boosting, Random Forests
than from expensive neural networks.

Three levels of ML Software

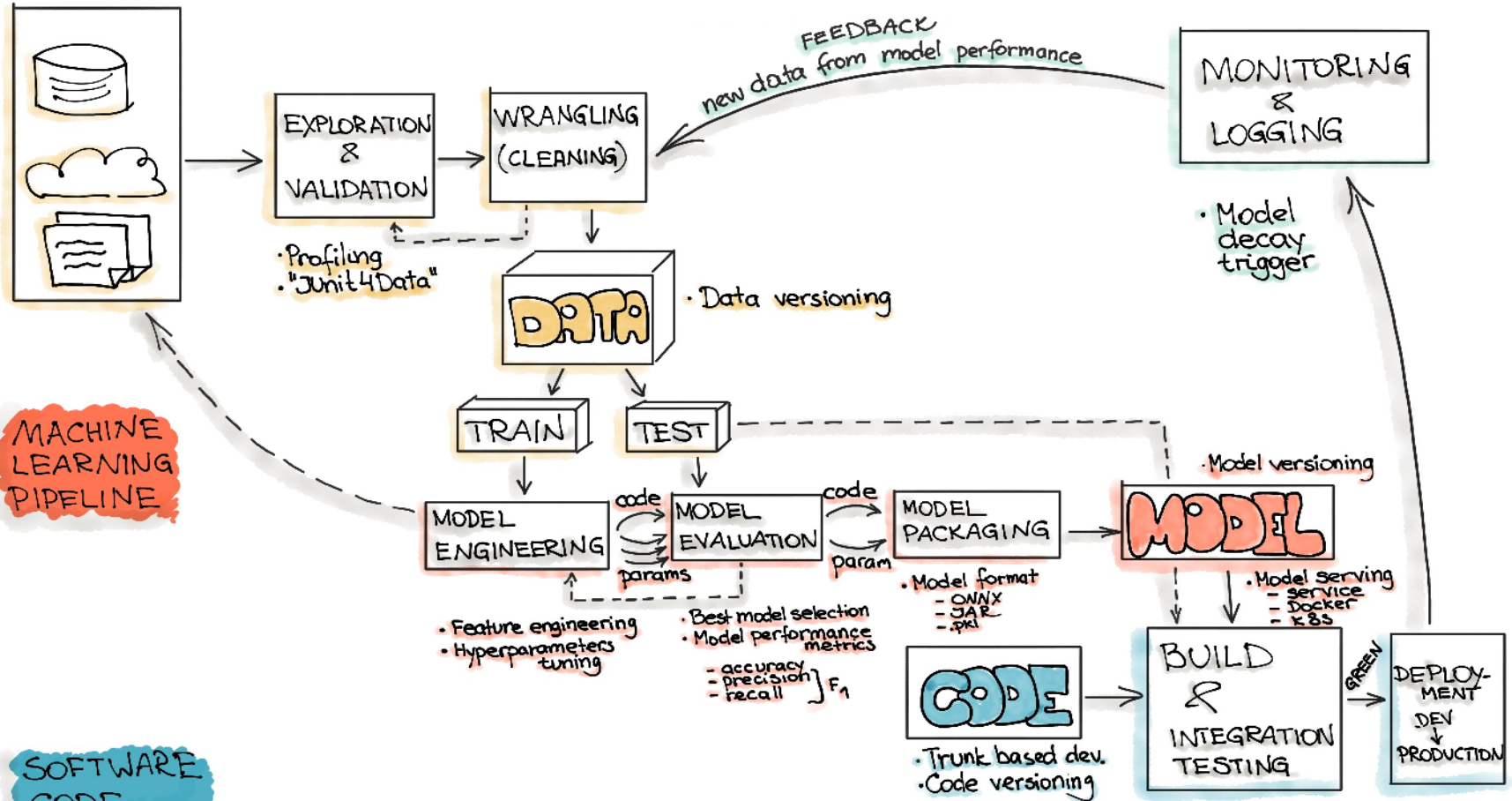
- ML/AI is rapidly adopted by new applications and industries!
- Goal of a machine learning project is to build a statistical model by using collected data and applying machine learning algorithms
 - Yet building successful ML-based software projects is still difficult
- Every ML-based software needs to manage three main assets:
 - Data
 - Model
 - and Code
- Essential technical methodologies - involved in the development of the Machine Learning-based software
 - Data Engineering: data acquisition & data preparation,
 - ML Model Engineering: ML model training & serving, and
 - Code Engineering : integrating ML model into the final product.

- Explainable AI
- Interpretability
- Model should be promoted by data scientists/ mlops to be used by stake holders
 - non technical explanation, feature selection, ground truth, human in the loop
 - More explainable (high interpretability)
 - Neural network (black box)
 - High interpretable -- > low performance
 - Low interpretability → high performance

Machine Learning Engineering

MACHINE LEARNING ENGINEERING

DATA PIPELINE



Machine Learning Engineering

Data → Vendors (missing values, duplicates, schema check, outliers)

Data providers

Clean data - > data engineering (data pipelines- ETL) zip file/ xml file -- > data lake

Transformation → features deletion/addition

Data Scientist → feature selection, Optimization parameters

Feature selection – 1 billion X 400 (shape)

- Features which are going to help in predicting the target variable
- Corr()
- Independent features (input features), independent variables
- Dimensionality reduction 1B X 50 (important features)

Machine Learning Engineering

Model training → model learns hidden pattern , mapping function from input variables (49) to the target feature

- Training phase – Are my features have any new information for the model to learn (20 % new)
- Threshold – correlation of 60% or more , I want to drop those features

Optimization → parameters

Machine Learning Engineering

Model → QA (unit testing) --> prod → UAT – User Acceptance Testing → deployed

Model monitoring

- time complexity
- failure rate (how many time fail/ total runs)
- change in the input data (data drift)
- drop in the performance

2 types

- scheduled jobs (10 AM , 4 PM, 10 PM) – CICD pipeline (group of components 1 ->2 ->3->4->5)
- even based triggers (Business team, data provider → user drop → datalake→folder)

Ground Truth – Actual Data -- > 03052025 → folder creation –event

Data Engineering

- The initial step in any data science workflow is to **acquire and prepare the data to be analyzed**
 - Typically, data is being integrated from **various resources** and has **different formats**
- Data Preparation
 - according to Gartner “**an iterative and agile process for exploring, combining, cleaning and transforming raw data into curated datasets for data integration, data science, data discovery and analytics/business intelligence (BI) use cases**”.
 - is reported to be the most expensive with respect to resources and time
- Data preparation is a critical activity in the data science workflow
 - important to avoid the propagation of data errors to the next phase, data analysis
 - would result in the derivation of wrong insights from the data

Data Engineering(2)

Sequence of operations on the data leading to training and testing datasets for ML algorithms

Data Ingestion

- Collecting data by using various frameworks and formats, such as Spark, HDFS, CSV, etc
- Might also include synthetic data generation or data enrichment.

Data Exploration

- Includes data profiling to obtain information about the content and structure of the data
- The output of this step is a set of metadata, such as max, min, avg of values

Data Validation

- Operations are user-defined error detection functions, which scan the dataset in order to spot some errors

Data Engineering(2)

Data Wrangling (Cleaning)

- The process of re-formatting particular attributes and correcting errors in data, such as missing values imputation

Data Labeling

- The operation of the Data Engineering pipeline, where each data point is assigned to a specific category

Data Splitting

- Splitting the data into training, validation, and test datasets to be used during the core machine learning stages to produce the ML model

Model Engineering

The core of the ML workflow

Phase of writing and executing machine learning algorithms to obtain an ML model

The Model Engineering pipeline includes a number of operations that lead to a final model:

Model Training

- The process of applying the machine learning algorithm on training data to train an ML model
- includes feature engineering and the hyperparameter tuning for the model training activity

Model Evaluation

- Validating the trained model to ensure it meets original codified objectives before serving the ML model in production to the end-user

Model selection

- Data-- > Imbalanced data
 - 100,000 records of credit card fraudulent transaction (historical data)
 - Target variable/ label/ Dependent Variable/Output Variable
 - Fraudulent Transaction (Yes/No)
 - 85000 – No
 - 15000 – yes (imbalanced data)
 - Balance data -- > Logistics regression

Ensemble Models -> can handle imbalanced data (RF, XGB)

Model Deployment

Need to deploy model as part of a business application such as a mobile or desktop application

- The ML models require various data points (feature vector) to produce predictions
- The final stage of the ML workflow is the integration of the previously engineered ML model into existing software

Includes the following operations:

Model Serving

- The process of addressing the ML model artifact in a production environment

Model Deployment

Model Performance Monitoring

- The process of observing the ML model performance based on live and previously unseen data, such as prediction or recommendation
 - interested in ML-specific signals, such as prediction deviation from previous model performance
 - signals might be used as triggers for model re-training
- Model Performance Logging
 - Every inference request results in the log-record

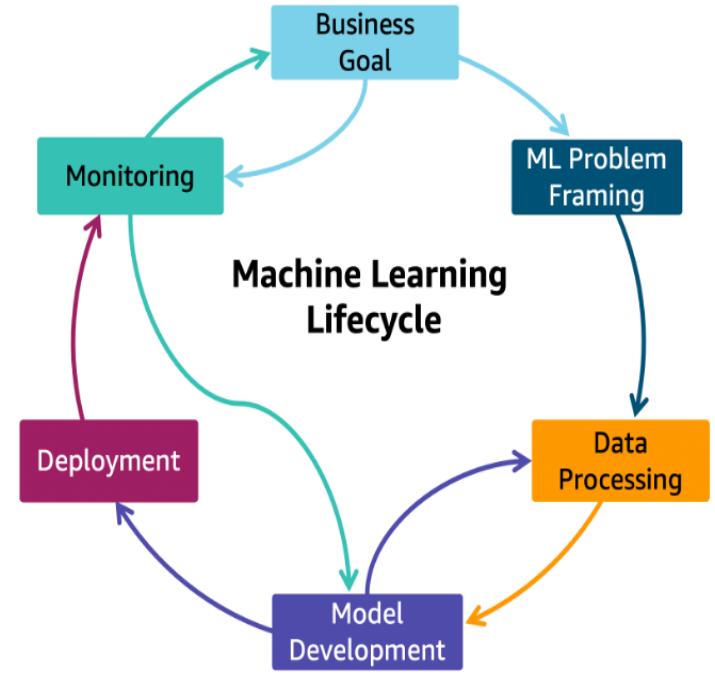
Machine Learning Lifecycle

Cyclic iterative process with instructions, and best practices to use across defined phases while developing an ML workload

- adds clarity and structure for making a machine learning project successful

The end-to-end machine learning lifecycle process includes the following phases:

- Business goal identification
- ML problem framing
- Data processing (data collection, data preprocessing, feature engineering)
- Model development (training, tuning, evaluation)
- Model deployment (inference, prediction)
- Model monitoring



Machine Learning Lifecycle(2)

Data processing

- Training an accurate ML model requires data processing to convert data into a usable format
- includes collecting data, preparing data
- and feature engineering that is the process of creating, transforming, extracting, and selecting variables from data

Model development

- consists of model building, training, tuning, and evaluation
- includes creating a CI/CD pipeline that automates the build, train and release to staging and production environments

Deployment

- After a model is trained, tuned, evaluated and validated, one can deploy the model into production
- can then make predictions and inferences against the model

Monitoring

- ensures model is maintaining a desired level of performance through early detection and mitigation

ML lifecycle phase — Business goal

- The most important phase particularly challenging when considering ML solutions because ML is a constantly evolving technology!
- After success criteria is defined, evaluate organization's ability to move toward that target
 - The target should be **achievable** and provide a clear path to production
 - Involve
 - all relevant stakeholders from the beginning to align them to this target
 - any new business processes that will result from this initiative

ML lifecycle phase — Business goal

Steps in this phase:

- Understand business requirements
- Form a business question
- Review a project's ML feasibility and data requirements
- Evaluate the cost of data acquisition, training, inference, and wrong predictions
- Review proven or published work in similar domains, if available
- Determine key performance metrics, including acceptable errors
- Define the machine learning task based on the business question
- Identify critical, must have features
- Design small, focused POCs to validate all of the preceding
- Evaluate if bringing in external data sources will improve model performance
- Establish pathways to production
- Consider new business processes that may come out of this implementation
- Align relevant stakeholders with this initiative

ML lifecycle phase — ML problem framing

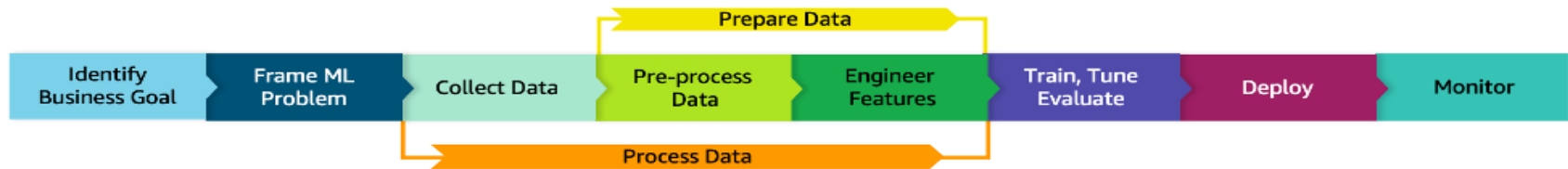
Business problem is framed as a machine learning problem

- what is observed and what should be predicted (known as a label or target variable)
- Determining what to predict and how performance must be optimized is a key step in ML

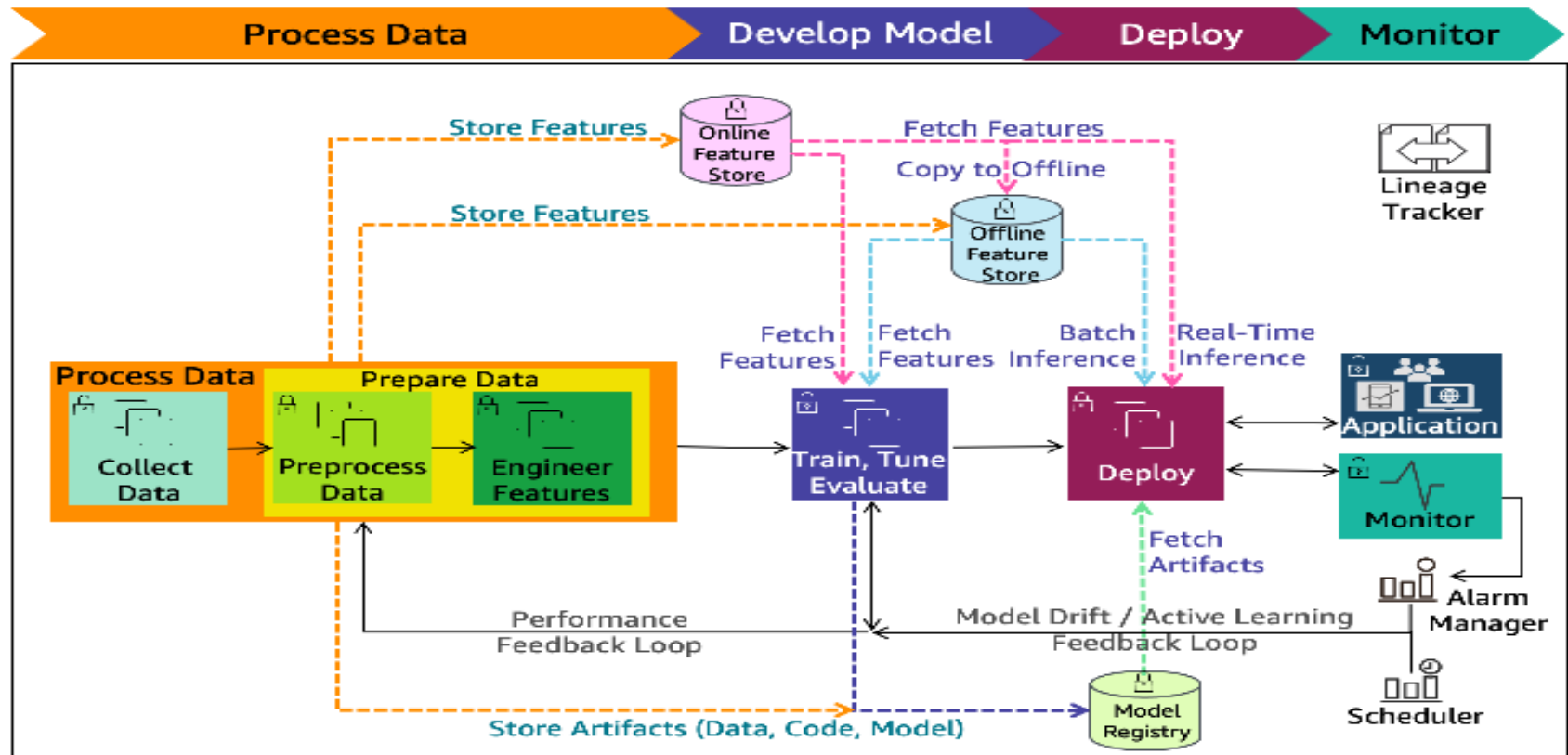
Steps in this phase:

- Define criteria for a successful outcome of the project
- Establish an observable and quantifiable performance metric for the project, such as accuracy
- Help ensure business stakeholders understand and agree with the defined performance metrics
- Formulate the ML question in terms of inputs, desired outputs, and the performance metric to be optimized
- Evaluate whether ML is the right approach
- Some business problems don't need ML, simple business rules can do a much better job
- For other business problems, there might not be sufficient data to apply ML as a solution
- Create a strategy to achieve the data sourcing and data annotation objective
- Start with a simple model that is easy to interpret, and which makes debugging more manageable

ML lifecycle architecture diagram ML lifecycle with data processing sub-phases included



ML lifecycle with data processing sub-phases included



ML lifecycle with detailed phases and expanded components

ML lifecycle architecture Components

Online/Offline feature store

- reduces duplication and rerun of feature engineering code across teams and projects
- Online store with low-latency retrieval capabilities is ideal for real-time inference
- Offline store should maintain a history of feature values and is suited for training and batch scoring

Model registry

- A repository for storing ML model artifacts including trained model and related metadata (data, code, model)
- enables lineage for ML models as it can act as a version control system

Performance feedback loop

- Automates model performance evaluation tasks initiated from the model development to data processing phase

ML lifecycle architecture diagram(4)

Model drift feedback loop

- Automates model update re-training tasks initiated from the production deployment to data processing phase

Alarm manager

- receives the alerts from the model monitoring system
- runs actions by publishing notifications to services that can deliver alerts to target applications to handle them
- The model update re-training pipeline is one such target application

Scheduler

- A scheduler can initiate a re-training at business defined intervals

Lineage tracker

- enables reproducible machine learning experiences
- enables re-creating the ML environment at a specific point-in-time, reflecting the versions of all resources and environments at that time
- collects references to traceable data, model and infrastructure resource changes

ML lifecycle phase - Data processing

Need

In ML workloads, the data (inputs and corresponding desired output) serves important functions including:

- Defining the goal of the system: the output representation and the relationship of each output to each input, by means of input/output pairs
- Training the algorithm that associates inputs to outputs
- Measuring the performance of the trained model, and evaluating whether the performance target was met
- Building baselines to monitor the performance of the models deployed to production
- Data processing includes data collection and data preparation
 - Data preparation includes data preprocessing and feature engineering
 - Data wrangling is data preparation during the interactive data analysis and model development
 - Data Visualization can help with exploratory data analysis (EDA)
 - EDA can help with understanding data, sanity checks, and validating the quality of the data
- The same sequence of data processing steps that you apply to the training data is also applied to the inference requests.

ML lifecycle phase - Data processing components

Data Processing

Data Collection

Data Collection

Label

Ingest (Streaming, Batch)

Aggregate

Data Preparation (Wrangling during Interactive Data Analysis) Leverage Visualization for Exploratory Data Analysis (EDA)

Data Preprocessing

Clean (Replace, Impute,
Remove Outliers, Duplicates)

Partition (Train, Validate, Test)

Scale(Normalize, Standardize)

Unbias, Balance
(Detection & Mitigation)

Augment

Feature Engineering

Feature Selection

Feature Transformation

Feature Creation
(Encoding, Binning)

Feature Extraction
(Automated in Deep Learning)

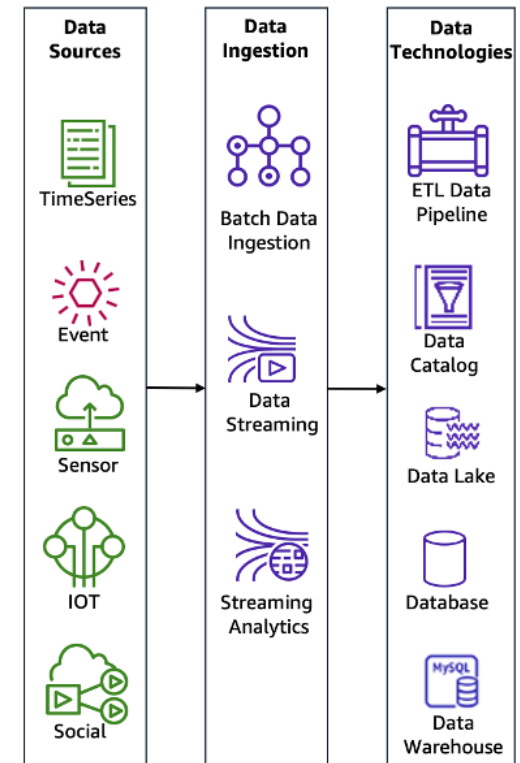
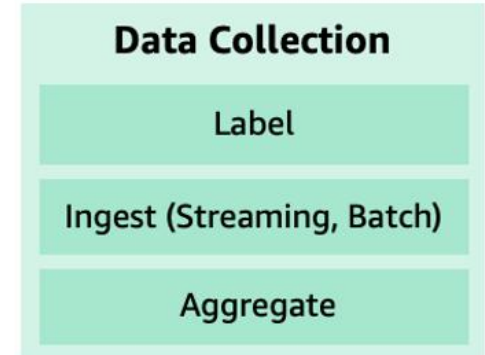
ML lifecycle phase - Data processing -Data collection

One of the first steps in the ML lifecycle is to identify what data is needed

- Then evaluate the various means available for collecting that data to train model

Activities in the data collection phase:

- Label — Data for which already know the target answer is called labeled data
 - If labels are missing, then some effort is required to label it (manual or automated)
- Ingest & Aggregate
 - Data collection includes ingesting and aggregating data from multiple data sources



ML lifecycle phase - Data processing -Data collection

Some components of the ingest and aggregate:

- Data sources
 - include time-series, events, sensors, IoT devices, and social networks, depending on the nature of the use case
- Data ingestion
 - processes and technologies capture and store data on storage media
 - can occur in real-time using streaming technologies or historical mode using batch technologies
- Data technologies
 - Data storage technologies vary from transactional (SQL) databases, to data lakes and data warehouses
 - ETL pipeline technology automates and orchestrates the data movement and transformations across cloud services and resources
 - A data lake technology enables storing and analyzing structured and unstructured data

ML lifecycle phase - Data preparation

- ML models are only as good as the data that is used to train them
 - Ensure that suitable training data is available and is optimized for learning and generalization

Data preparation includes

- data preprocessing
- feature engineering

A key aspect to understanding data is to identify patterns which are often not evident with data in tables

- Exploratory data analysis (EDA) with visualization tools can help quickly gain a deeper understanding of data
- Prepare data using wrangler tools for interactive data analysis and model building
- The no-code/low-code, automation, and visual capabilities improve the productivity and reduces the cost for interactive analysis

ML lifecycle phase - Data preprocessing

Data preprocessing puts data into the right shape and quality for training

- Many strategies: data cleaning, balancing, replacing, imputing, partitioning, scaling, augmenting and unbiasing

Clean (replace, impute, remove outliers and duplicates)

- Remove outliers and duplicates, replace inaccurate or irrelevant data
- Correct missing data using imputation techniques that will minimize bias as part of data cleaning

Partition

- To prevent ML models from overfitting and evaluate trained model accurately, randomly split data into train, validate, and test sets
- Care is needed to avoid data leakage
- Data leakage happens when information from hold-out test dataset leaks into the training data
- One way to avoid data leakage is to remove duplicates before splitting of the data

Data Preprocessing

Clean (Replace, Impute, Remove Outliers, Duplicates)

Partition (Train, Validate, Test)

Scale(Normalize, Standardize)

Unbias, Balance
(Detection & Mitigation)

Augment

ML lifecycle phase - Data preprocessing

Scale (normalize, standardize)

- Having features on a similar scale close to normally distributed will ensure that each feature is equally important
- make it easier for most ML algorithms including K-Means, KNN, PCA, gradient descent
- Standardization better handles the outliers

Unbias, balance (detection & mitigation)

- Detecting and mitigating bias will help avoiding inaccurate model results
- Biases are imbalances in the accuracy of predictions across different groups, such as age or income bracket
- Biases can come from the data or algorithm used to train your model

• Augment

- increases the amount of data artificially by synthesizing new data from existing data
- can help regularize and reduce overfitting

ML lifecycle phase -Feature engineering

Every unique attribute of the data is considered a feature

Feature engineering is a process to select and transform variables when creating a predictive model using machine learning or statistical modeling

- includes feature creation, feature transformation, feature extraction, and feature selection
- With deep learning, the feature engineering is automated as part of the algorithm learning

Feature creation

- is creating new features from existing data to help with better predictions
- Examples of feature creation techniques include: one-hot-encoding, binning, splitting, and calculated features

Feature Engineering

Feature Selection

Feature Transformation

Feature Creation
(Encoding, Binning)

Feature Extraction
(Automated in Deep Learning)

ML lifecycle phase -Feature engineering

Feature transformation and imputation

- manage replacing missing features or features that are not valid
- Some techniques include: forming Cartesian products of features, non-linear transformations (such as binning numeric variables into categories), and creating domain-specific features

Feature extraction

- involves reducing the amount of data to be processed using dimensionality reduction techniques
- reduces the amount of memory and computing power required, while still accurately maintaining original data characteristics
- Techniques include: PCA, ICA, and LDA

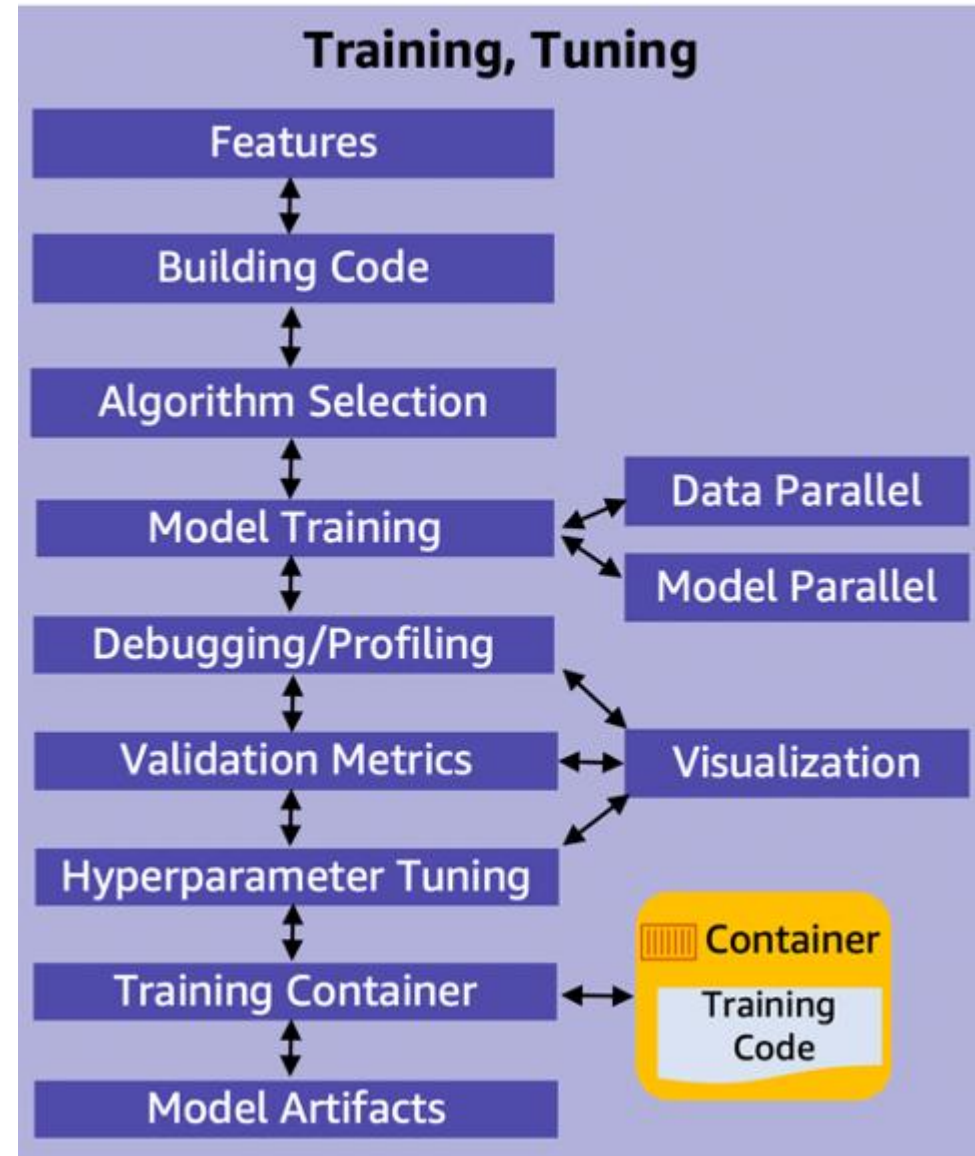
Feature selection

- process of selecting subset of extracted features which is relevant and contributes to minimizing the error rate of a trained model
- Feature importance score and correlation matrix can be factors in selecting the most relevant features for model training

ML lifecycle phase – Model development -Model building, training, tuning

In this phase, select a machine learning algorithm that is appropriate for problem and then train the ML model

- provide the algorithm with the training data
- set an objective metric for the ML model to optimize on
- and set the hyperparameters to optimize the training process



ML lifecycle phase – Model development -Model training, tuning activities

Features selection

- Features are selected as part of the data processing after a bias strategy is implemented

Building code

- Model development includes building the algorithm and its supporting code
- should support version control, and continuous build, test, and integration through a pipeline

Algorithm selection

- Selecting the right algorithm involves running many experiments with parameter tunings across available options
- Factors to consider when evaluating each option can include accuracy, explainability, training/prediction time, memory requirements

ML lifecycle phase – Model development -Model training, tuning activities

Model training (data training parallel, model training parallel)

- The process of training an ML model involves providing an ML algorithm with training data to learn from
- Distributed training enables splitting large models and training datasets across computing instances to reduce runtime to fraction of it takes to do manually
- Model parallelism is the process of splitting a model up between multiple devices or nodes.
- Data parallelism is the process of splitting the training set in mini-batches evenly distributed across nodes
 - each node only trains the model on a fraction of the total dataset

ML lifecycle phase – Model development(3)

Debugging/profiling

- A machine learning training job can have problems including:
 - system bottlenecks, overfitting, saturated activation functions, and vanishing gradients. These problems can compromise model performance
 - A debugger provides visibility into the ML training process through monitoring, recording, and analyzing data
 - captures the state of a training job at periodic intervals
- Validation metrics
 - Typically, a training algorithm computes several metrics, such as loss and prediction accuracy
 - determine if the model is learning and generalizing well for making predictions on unseen data
 - Metrics reported by the algorithm depend on the business

ML lifecycle phase – Model development(3)

Debugging/profiling

- A machine learning training job can have problems including:
 - system bottlenecks, overfitting, saturated activation functions, and vanishing gradients. These problems can compromise model performance
 - A debugger provides visibility into the ML training process through monitoring, recording, and analyzing data
 - captures the state of a training job at periodic intervals
- Validation metrics
 - Typically, a training algorithm computes several metrics, such as loss and prediction accuracy
 - determine if the model is learning and generalizing well for making predictions on unseen data
 - Metrics reported by the algorithm depend on the business problem and the ML technique used

ML lifecycle phase – Model development(4)

ML lifecycle with pre-production pipelines

- The data prepare pipeline automates data preparation tasks
- The feature pipeline automates the storing, fetching, and copying of the features into and from online/offline store
- The CI/CD/CT pipeline automates the build, train, and release to staging and production environments

ML lifecycle phase – Model development(5)

Model evaluation

- After the model has been trained, evaluate it for its performance and accuracy
 - generate multiple models using different methods and evaluate the effectiveness of each model
 - for multiclass models, determine error rates for each class separately
- Can evaluate model using historical data (offline evaluation) or live data (online evaluation)
- Offline evaluation
 - the trained model is evaluated with a portion of the dataset that has been set aside as a holdout set
 - never used for model training or validation—it's only used to evaluate errors in the final model

ML lifecycle phase – Model development(6)

ML lifecycle with performance evaluation pipeline added

ML lifecycle phase - Deployment

- After model have been trained, tuned, and evaluated, then can deploy the model into production
 - can make predictions and inferences against the deployed model
 - Use a manual governance process - ensures the model has fully been tested and evaluated before it's released to production
- Deploy production phase of the ML lifecycle
 - Features are retrieved from the feature store
 - Artifacts are taken from the model registry
 - Inference code used container from the container repository

ML lifecycle phase – Deployment(2)

Deployment main components

- Blue/green, canary, A/B, shadow deployment/testing
 - Deployment and testing strategies that reduce downtime and risks when releasing a new or updated version
- The blue/green deployment technique
 - provides two identical production environments
 - can use this technique when need to deploy a new version of the model to production
 - Once testing is done on the green environment, live application traffic is directed to the green environment
 - Blue environment is maintained as backup
- A canary deployment
 - first deploy the new release to a small group of users
 - Other users continue to use the previous version until satisfied

ML lifecycle phase – Deployment(3)

ML lifecycle with scheduler re-train, and batch/real-time Inference pipelines

- Inference pipeline
 - automates capturing of the prepared data, performing predictions and post-processing for real-time or batch inferences
- Scheduler pipeline
 - Deployed model is representative of the latest data patterns
 - Re-training at intervals can minimize the risk of data and concept drifts
 - A scheduler can initiate a re-training at business defined intervals
 - Data prepare, CI/CD/CT, and feature pipelines will also be active during this process

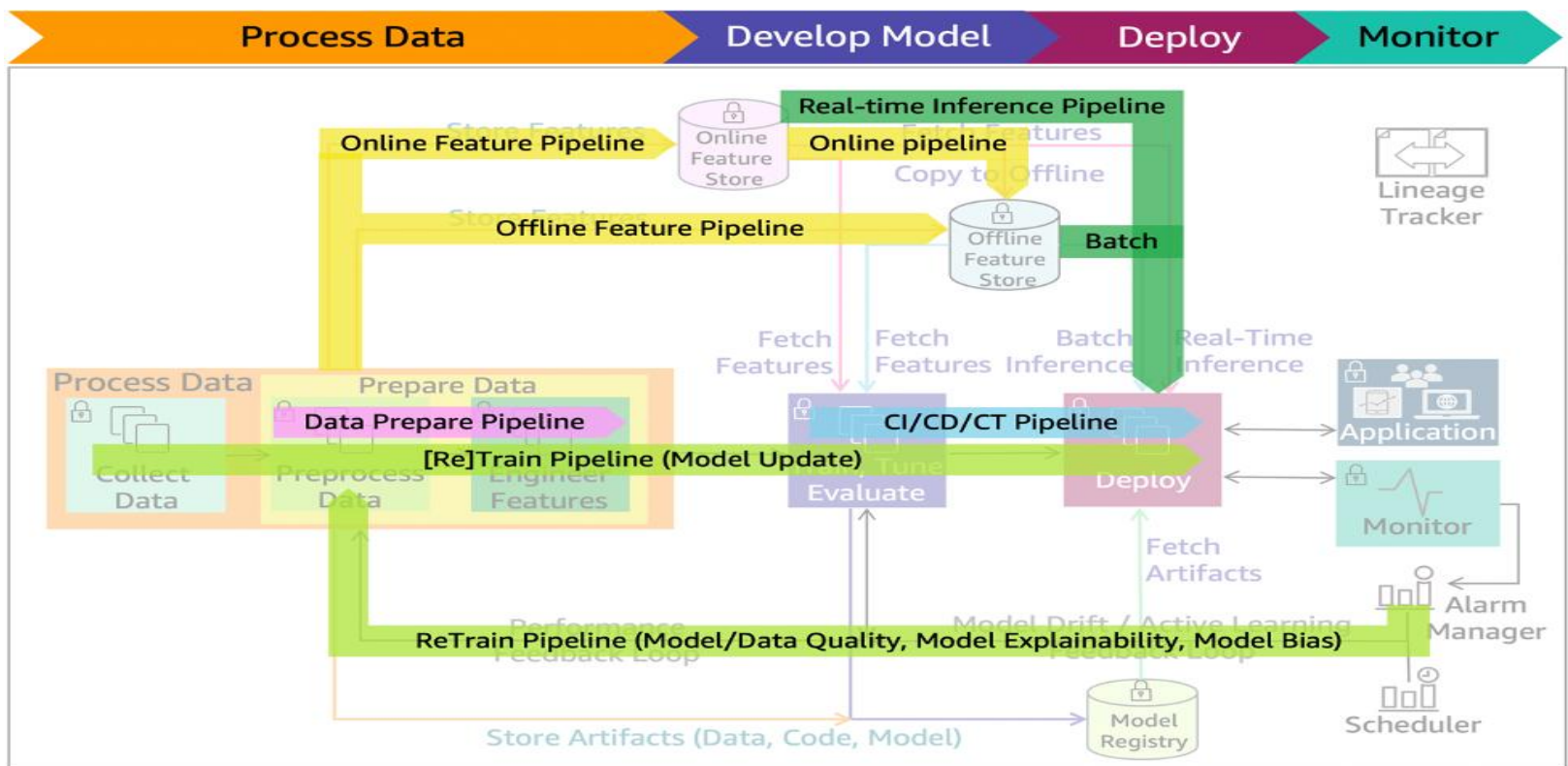
ML lifecycle phase – Monitoring

Post deployment monitor main components

- The model monitoring system must
 - capture data,
 - compare that data to the training set, define rules to detect issues
 - and send alerts
- Process repeats on a defined schedule, when initiated by an event, or when initiated by human intervention
 - The issues detected in the monitoring phase include: data quality, model quality, bias drift, and feature attribution drift
- Key components of monitoring including:
 - Model explainability
 - uses explainability to evaluate the soundness of the

ML lifecycle phase – Monitoring(2)

ML lifecycle with model update re-train and batch/real-time Inference pipelines



Thank You!

In our next session:



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

ML System Architecture

Adapted from “**Amit Kumar**
Architecture
of a real-world Machine
Learning system” by Louis
Dorard

ML System Architecture

Client-server
architecture of a
“supervised
learning” system

‘entry points’

the client requesting predictions, and the orchestrator creating/updating models

- Client represents the application
 - used by the end-user who will benefit from the ML system
 - can be the smartphone app used to order dinner
- Orchestrator would usually be a program called by a scheduler
 - so models could be updated periodically, e.g., every week
 - or called via an API (so that it could be part of a CI/CD pipeline)
 - In charge of evaluating models made by the model builder, on a test dataset that is kept secret
 - sends test predictions to the evaluator

Monitoring

- Assuming one or several (baseline) models would be available as APIs,
 - but would not be integrated into the final application yet,
 - need decide which model to integrate (and whether it's safe) by tracking performance on production data and visualizing it via a monitor
- In dinner-delivery example,
 - would let you compare a model's ETD with the actual time of delivery, on orders that were just delivered
- When a new model version becomes available,
 - client requests for predictions would be progressively directed to the new model's API, via the front-end
 - would be done for an increasing number of end-users while monitoring performance and checking that the new model is not “breaking” anything

All the components

- Ground-truth Collector
- Data Labeller
- Evaluator
- Performance Monitor
- Featurizer
- Orchestrator
- Model Builder
- Model Server
- Front-end

#1: GROUND-TRUTH COLLECTOR

In the real world, it is key to be able to continuously acquire new data for the machine to learn from

- Ground-truth data
 - corresponds to what you want your ML models to predict, such as
 - the sale price of real-estate property,
 - a customer-related event (e.g. churn),
 - or a label to assign to input objects (e.g. ‘spam’ on incoming messages)
- Sometimes, you observe an input object, need to wait for a certain amount of time to observe the thing you wanted to predict about this object
 - wait for the property to get sold,
 - wait for the customer to renew or cancel their subscription,

#2: DATA LABELLER

- Sometimes, have access to plenty of input data, but need to create the associated ground-truth data manually
 - case when building a spam detector, or an object detector from images
- Ready-made and open-source web apps to make data labeling easier
 - such as Label Studio
- Dedicated services for outsourcing the manual task of labeling data
 - for example, Figure Eight and Google's Data Labeling Service

#3: EVALUATOR

- Before setting out to build any ML models with an initial dataset for the machine to learn from
 - it's important to define how to evaluate the planned ML system
- In addition to measuring prediction accuracy,
 - evaluation of short-term and long-term impact via application-specific performance metrics,
 - and system metrics such as lag and throughput, are desirable
- Two important objectives behind model evaluation:
 - comparing models
 - deciding whether it is safe to integrate a model into an application
- Evaluation can be performed on a predetermined set of test cases, for which it is known what the prediction

#4: PERFORMANCE MONITOR

- The next step towards deciding if a (baseline) model can be integrated into an application is
 - to use it on the inputs encountered in production (called ‘production data’), in a production-like setting,
 - and to monitor its performance through time
- Computing and monitoring performance metrics on production data requires
 - acquiring and storing production inputs, ground truths, and predictions in a database
- The performance monitor would consist of a program that
 - reads from that database,
 - calls the evaluator,
 - and updates a dashboard that shows how performance metrics evolve with time

#5: FEATURIZER

- When designing a prediction API, a decision needs to be made as to what the API should take as input
 - For example,
 - when making predictions about customers, should the input be the full feature representation of the customer, or just the customer id?
- Common that the full numerical representation would not be readily available
 - but it would have to be computed before it can be passed to a model
- For a customer input, some features would be already stored in a database
 - for example, date of birth
- Others would require computation-

○ For example, how the customer interacted with the product

#5: FEATURIZER(2)

- If, by nature, features do not change too often, they could be computed in batches
- Could have “hot” features that would change rapidly and would need to be computed in real-time
 - for instance, the average delivery time of a given restaurant over the last X minutes
- Calls for the creation of at least one featurization microservice
 - that would extract features for a batch of inputs, based on their ids
- May also need a real-time featurization microservice,
 - but this would be at an additional cost in the complexity of ML system
- Featurizers may query various databases and perform aggregations and treatments on the queried data

○ may have parameters (such as the number of minutes X) which

#6. ORCHESTRATOR

Workflow

- The orchestrator is at the core of the ML system and interacts with many other components
- Steps in its workflow/pipeline:
 - 1) Extract-Transform-Load and split (raw) data into training, validation, test sets
 - 2) Send training/validation/test sets for featurization (if any)
 - 3) Prepare featurized training/validation/test sets
 - 4) Send URIs of prepared train/validation sets, along with metric to optimize, to model builder
 - 5) Get optimal model, apply to test set, and send predictions

#6. ORCHESTRATOR

Ways to run the workflow

- The whole workflow could be executed manually
- Will have to be automated
 - If needs to update models frequently, or to tune hyperparameters of the featurizer and of the modeler jointly
- Workflow might be implemented as a simple script and run on a single thread, but computations could be more efficient by parallelizing runs.
 - End-to-end ML platforms allow doing that and can provide a single environment to define and run full ML pipelines

Thank You!

In our next session: